

STRANGE SUSHI

OOP PROJECT

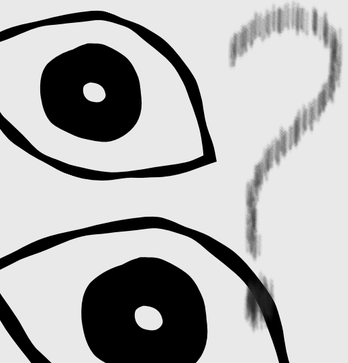
68160002 CHOTIKA THEPSORN



INTRODUCTION

แนวคิดและแรงบันดาลใจ
ผสมผสานระหว่าง

เกมจำลองการทำอาหาร + **เกมระทึกขวัญ**

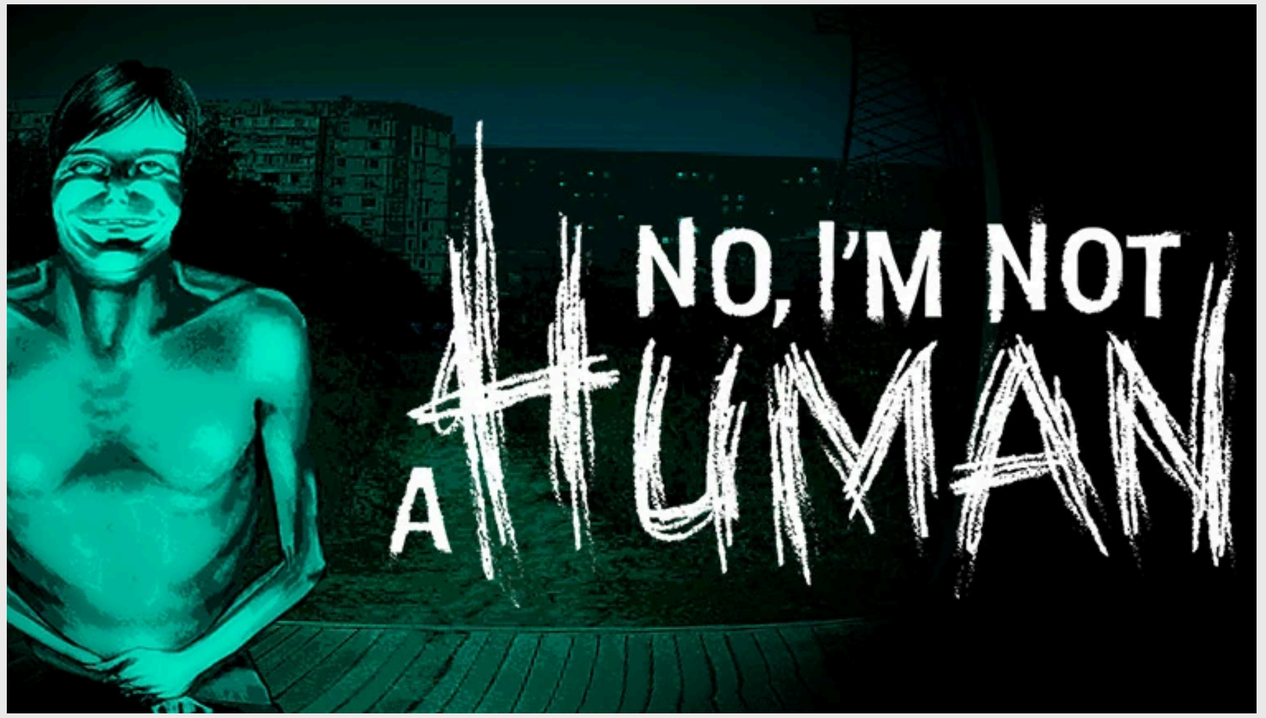


Gameplay

ผู้เล่นรับบทเป็นเจ้าของร้านซูชิ

- ต้องบริหารเงิน จ่ายค่าเช่า และเสิร์ฟเมนูให้ถูกต้องและทันเวลา
- จุดเด่น ลูกค้ามีทั้ง "คนปกติ" และ "ผี"



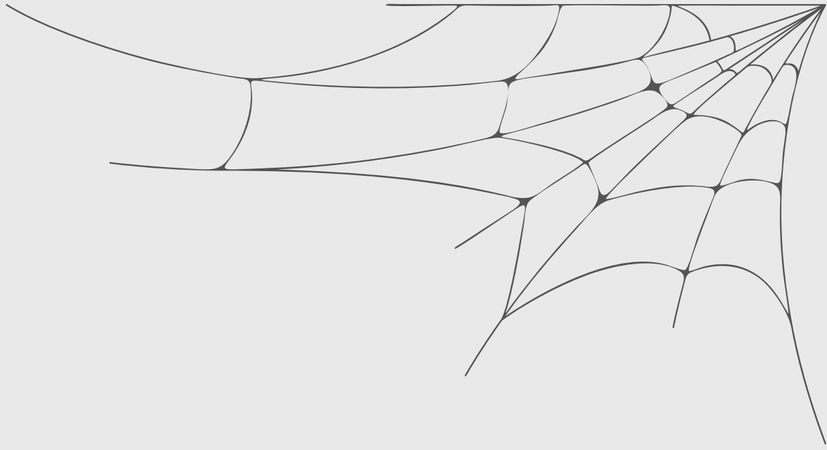




ได้รับแรงบันดาลใจจากบรรยากาศของเกม
‘No, I’m not human’



OBJECTIVES

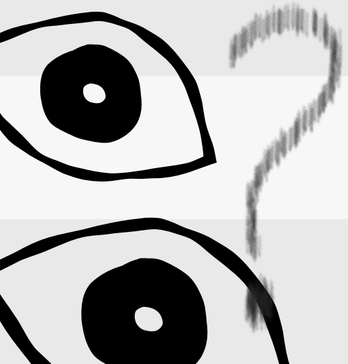


1. เพื่อฝึกฝนการใช้หลักการ OOP ในการพัฒนาโปรแกรมจริง

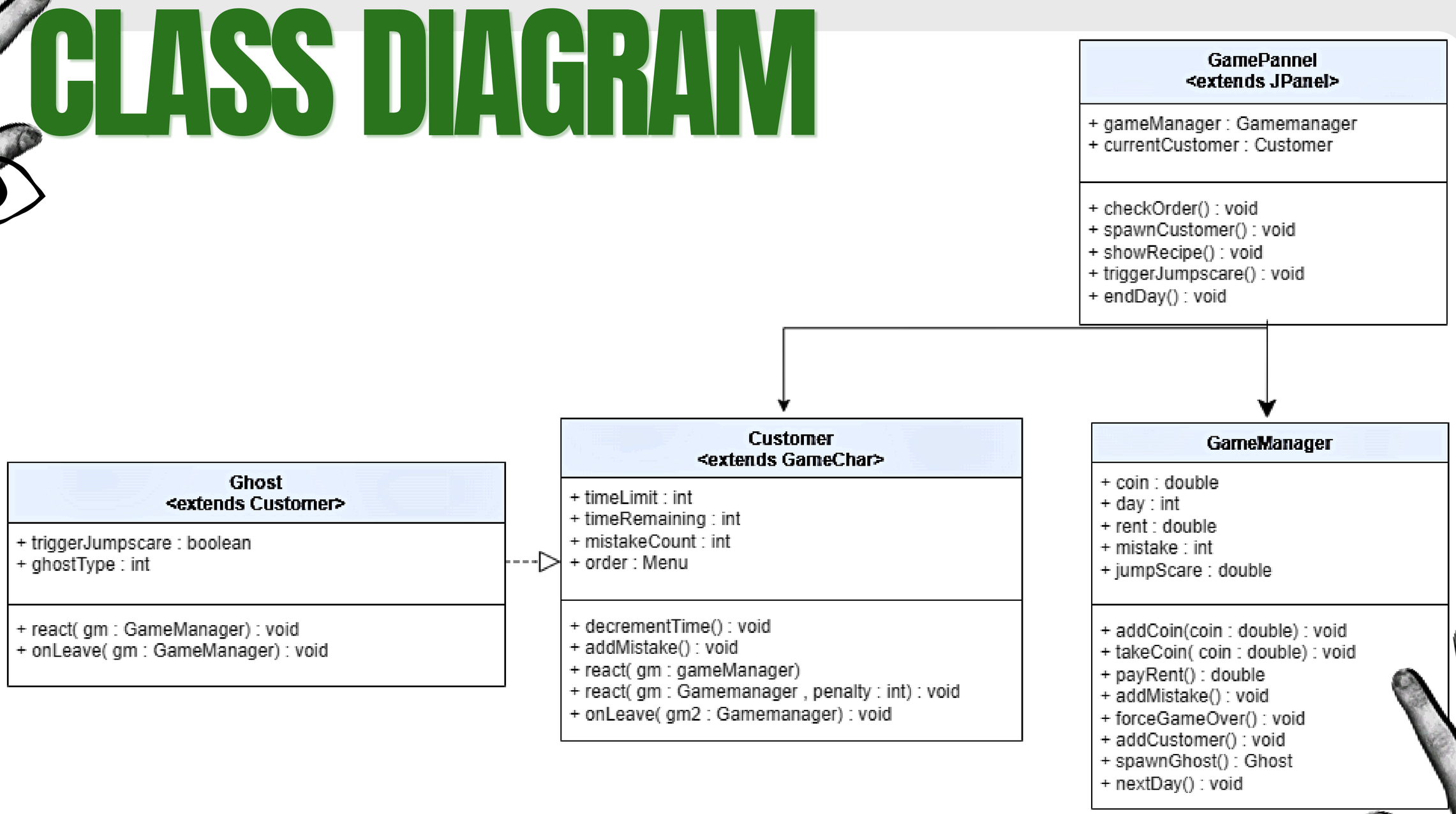
2. เพื่อออกแบบโครงสร้างโปรแกรมเกมอย่างเป็นระบบและมีประสิทธิภาพ

3. เพื่อประยุกต์ใช้ความรู้ด้านภาษา Java และไลบรารีพื้นฐาน

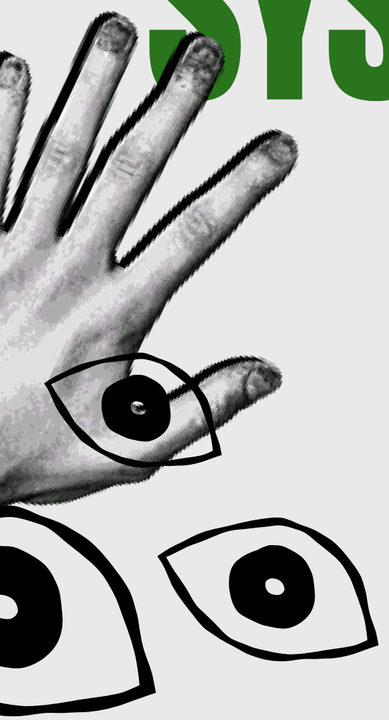
4. เพื่อทดลองสร้างและแก้ปัญหาในการพัฒนาระบบจากความรู้ที่ได้ศึกษา

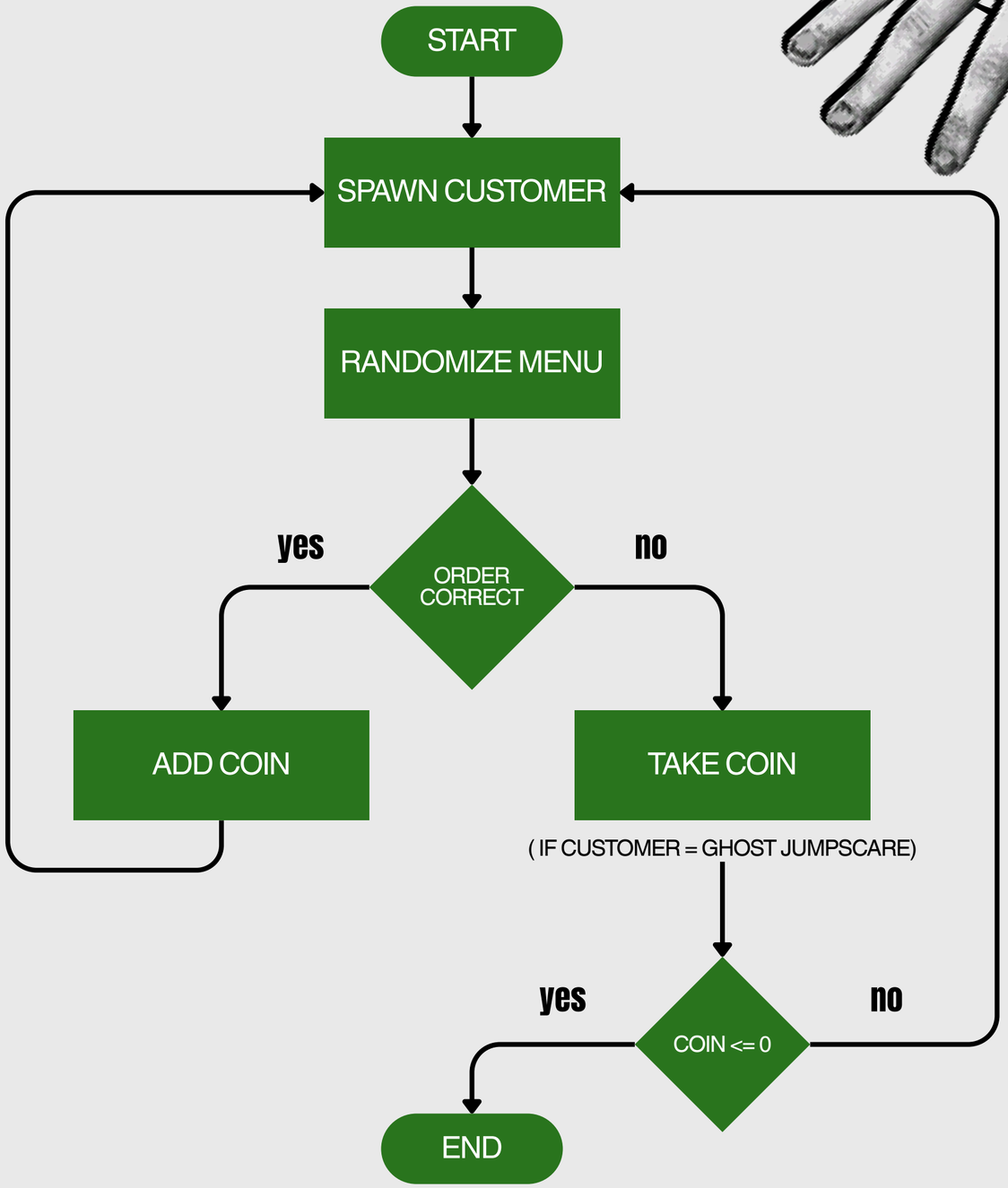


CLASS DIAGRAM



SYSTEM FLOW




START
SPAWN CUSTOMER
RANDOMIZE MENU
ORDER CORRECT
yes
no
ADD COIN
TAKE COIN
(IF CUSTOMER = GHOST JUMPSCARE)
COIN <= 0
yes
no
END

7 OOP CONCEPTS



Encapsulation

```
public class GameManager {  Chotika Thepsorn *
    private double coin;
    public double getCoin(){return coin;}  Chotika Thepsorn
    public void addCoin(double coin){ //add money if correct order  Chotika Thepsorn
            this.coin += coin;
    } // add coin
    public void takeCoin(double coin) {  Chotika Thepsorn
        this.coin -= coin;//take coin
        setGameOver(); //check game over
    }
```

Association

```
public class GamePannel extends JPanel { //to make GamePanel class panel
    private final Recipe recipe;              // for random dish orders
    private final GameManager gameManager;  // to talk to GameManager
```

Inheritance

/Class Ghost extends Customer

```
public class Ghost extends Customer  {   Chotika Thepsorn *
    private int ghostType;
    private String jumpscareImage;
    private boolean triggerJumpscare;
    private GameManager gm;

    //-------------------Constructor-------------------
    public Ghost(int ghostType, GameManager gameManager) {   Chotika Thepsorn
        super();
        this.gm = gameManager;
        this.ghostType = ghostType;
```

Abstract Class

/Class GameChar

```
public abstract class GameChar {  2 inheritors   Chotika Thepsorn
    public abstract void react(GameManager gm);  2 implementations
    //what each customer react when get wrong order
    public abstract void onLeave(GameManager gm);  2 implementations
    //what each customer do when timer run out
}
```

used by class Customer

overloading

Polymorphism

/Class Customer

```
@Override  1 override  Chotika Thepsorn *
public void onLeave(GameManager gm2 ) { gm2.takeCoin(45); };
```

/Class Ghost

```
//--------------------OnLeave--------------------
@Override  Chotika Thepsorn
public void onLeave(GameManager gm) { gm.forceGameOver(); }
```

override

Polymorphism

/Class Customer

```
@Override  1 override  Chotika Thepsorn
public void react(GameManager gm){ //when wrong order
    addMistake();
    gm.addMistake();
    gm.takeCoin(35);
};

public void react(GameManager gm, int penalty) { //more day = more penalty
    addMistake();
    gm.addMistake();
    gm.takeCoin(penalty);
}
```



STRANGE SUSHI

OOP PROJECT

CHOTIKA THEPSORN